



AI with Rav

Learn AI the way you actually think.



DAY 9 of 30

MACHINE LEARNING

XGBoost — learning from your own mistakes

WHAT YOU'LL LEARN TODAY

- › How XGBoost is different from Random Forest
- › Why "fixing mistakes" beats "voting"
- › The simple idea of boosting, step by step
- › Why it wins almost every competition



XGBoost — learning from your own mistakes

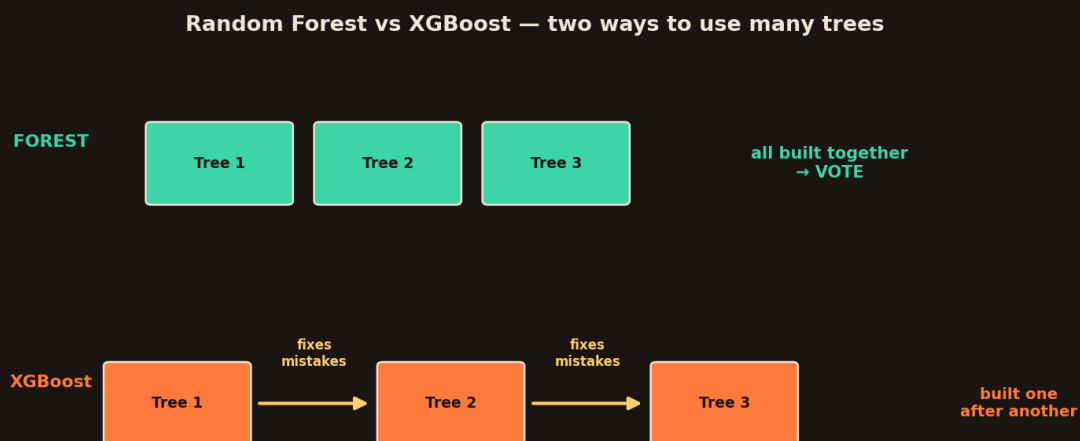
In One Line: XGBoost builds trees one after another — each new tree fixes the mistakes of the ones before it. It's like a student who only re-studies the questions they got wrong. That's why it wins competitions.

Start here: the smart student

Two students prepare for an exam. Student A studies the whole book again and again. Student B is smarter: after each practice test, she looks **only at the questions she got wrong** and fixes exactly those. Guess who scores higher?

Student B — fixing your mistakes beats repeating everything. XGBoost is Student B, turned into a machine. And on Kaggle (the world's biggest ML competition site), XGBoost wins more than almost any other algorithm.

Forest vs XGBoost — the key difference



Forest: all trees vote at once. XGBoost: each new tree fixes the mistakes of the ones before it.

Random Forest builds all trees together and votes. XGBoost builds trees one after another, each fixing the last one's mistakes.

Yesterday's Random Forest built many trees **at the same time** and let them **vote**. XGBoost does the opposite:

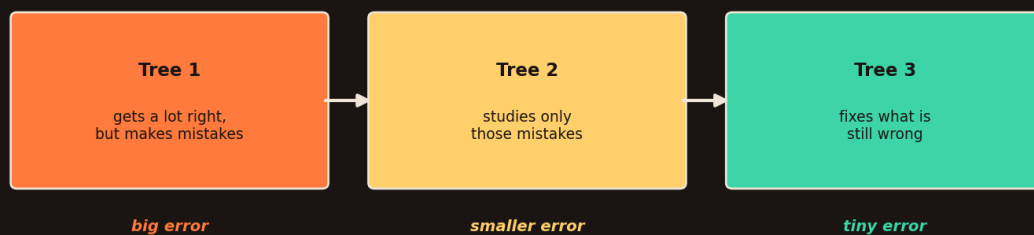
- It builds trees **one after another** (in a line, not side by side).
- Each new tree looks at what the earlier trees got **wrong**, and tries to fix just that.
- This "learn, then fix the mistakes, then learn again" style has a name: **boosting**.



Forest = a crowd voting all at once. XGBoost = a team where each new member fixes the last one's errors. Both use many trees — but XGBoost's "fix the mistakes" approach usually squeezes out more accuracy.

How boosting actually works

Each new tree studies the LEFTOVER mistakes



Add up all the trees → a super-accurate answer. Like a student who only re-studies the questions they got wrong.

Tree 1 makes big mistakes. Tree 2 studies only those mistakes. Tree 3 fixes what's still wrong. Add them up = a super-accurate answer.

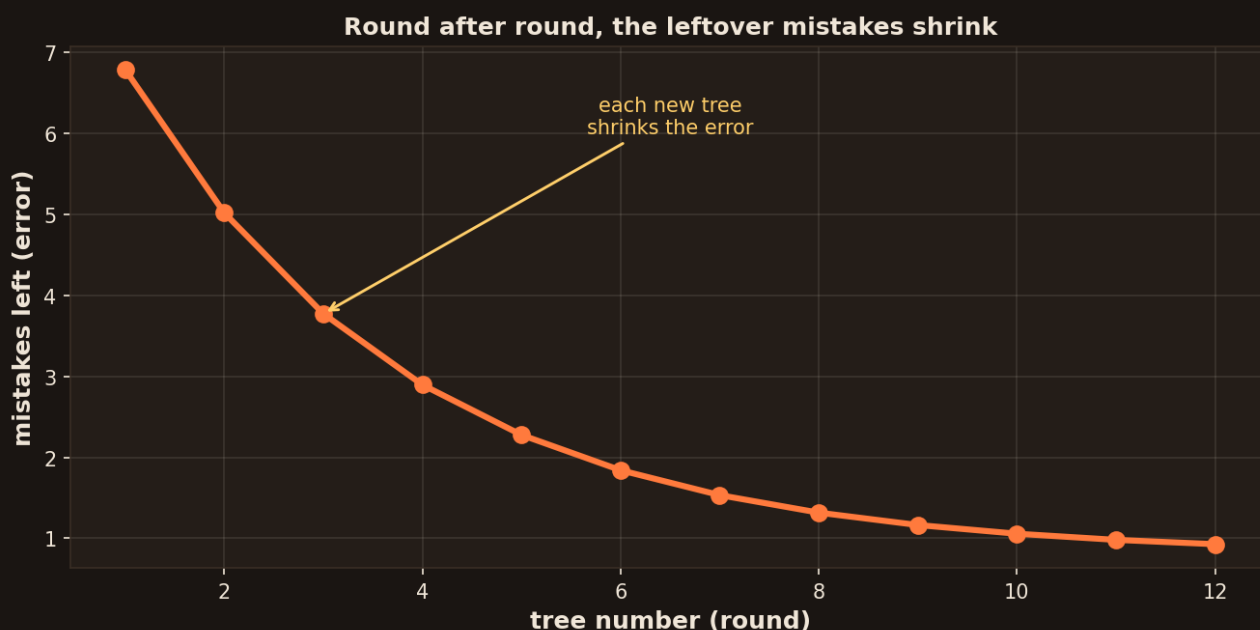
Think of the smart student again:

- **Tree 1** takes the "test" → gets a lot right, but makes some mistakes.
- **Tree 2** doesn't redo everything — it studies **only Tree 1's mistakes** and fixes them.
- **Tree 3** fixes whatever is **still** wrong after Tree 1 and Tree 2.
- Keep going... and you add up all the trees into one very sharp final answer.

Each tree is weak on its own — but each one cleans up a little more of the leftover mistake. Stack enough of them, and together they become extremely accurate. That's the power of learning from errors, step by step.

Watch the mistakes shrink





Round after round, the leftover mistakes get smaller and smaller. That's boosting in action.

Every round, the pile of "still wrong" gets smaller. The first few trees fix the big, obvious mistakes; the later trees polish the tricky little ones.

This is why XGBoost is so accurate: it doesn't stop at "good enough" — it keeps hunting down the leftover mistakes, one tree at a time, until very little is left. But careful — if it hunts too hard, it starts memorising (overfitting). That's why it has "brakes" (next section).

The formula — but really simple

No scary maths. Boosting is just **adding up trees**, where each tree corrects the last:

Final answer = Tree 1 + Tree 2 + Tree 3 + ... (each one a small fix on top of the last)

Two important "brakes" (settings) keep it from memorising:

- **Learning rate** → how big a step each new tree is allowed to fix. Small steps = safer, needs more trees. (Like studying slowly and carefully.)
- **Number of trees** → how many rounds of fixing. Too many = starts memorising.

That's the whole idea: keep adding small corrections. The "XG" in XGBoost just means "eXtreme Gradient" — a fast, clever way of choosing each fix. You don't need the heavy maths to use it well; you need to know it fixes mistakes step by step, with brakes.

Train one — a few lines



```

from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=200,      # how many trees (rounds of fixing)
    learning_rate=0.1,     # step size of each fix (the brake)
    max_depth=4           # keep each tree small
)
model.fit(X_train, y_train)

print(model.predict([[income, age, has_credit]]))
print(model.feature_importances_)  # which clues mattered most

```

Three knobs do most of the work: `n\_estimators` (how many fixes), `learning\_rate` (how big each fix), `max\_depth` (how deep each tree). Tune these and XGBoost often becomes the most accurate model on table data.

When to use it (and when not)

- **Best on table data** → rows and columns (fraud, sales, loans, risk). This is where XGBoost is king.
- **Needs a little tuning** → those knobs matter; a Random Forest works okay with almost no tuning, XGBoost rewards you for tuning it.
- **Not for images/text/speech** → deep learning wins there (coming later).
- **Can overfit if pushed too hard** → too many trees or too big a learning rate = it memorises. Use the brakes.

Recap — the 20-second version

- XGBoost builds trees **one after another**, each fixing the last one's **mistakes** (boosting).
- Like a student who **re-studies only the wrong answers** — that's why it's so accurate.
- Forest = vote all at once. XGBoost = fix mistakes step by step.
- Formula = just **adding trees**, with **brakes** (learning rate, number of trees) so it doesn't memorise.
- The **Kaggle king** for table data — fraud, loans, sales.

Next up — Video 10: K-Nearest Neighbours (KNN). The simplest idea in all of ML: "you are the average of your 5 closest friends." Judge something new by looking at who it sits near. See you Day 10.

